SMART BUDGET OPTIMIZER USING REACT & TYPESCRIPT

Project Report

Coding Skills - 1 (CSE201)

Bachelor of Technology
Computer Science and Engineering

Submitted by

M.K.VISWANATH-AP24110011220

SRM University–AP Neerukonda,
Mangalagiri, Guntur, Andhra Pradesh 522240

[Dec, 2025]

Table of Contents

Title Page i
Certificate 4
Acknowledgement 5
Abstract 6
CHAPTER 1 – INTRODUCTION 7
1.1 Introduction
1.2 Problem Statement
1.3 Objectives of the Project
CHAPTER 2 – SYSTEM ANALYSIS 8
2.1 Existing System
2.2 Proposed System
2.3 System Architecture / Block Diagram
CHAPTER 3 – SYSTEM DESIGN 9-10
3.1 Modules Description
3.1.1 Input Module
3.1.2 Validation Module
3.1.3 Greedy Algorithm Module
3.1.4 Dynamic Programming Module
3.1.5 Comparison Module
3.1.6 Results & Visualization Module
3.2 Data Flow Diagram (DFD)
3.3 System Flowchart
CHAPTER 4 – IMPLEMENTATION 11-22
4.1 Project Requirements (Software + Hardware)
4.2 Code Implementation
4.3 File Structure & Storage Format
CHAPTER 5 – RESULTS 23-27
CHAPTER 6 – CONCLUSION & FUTURE WORK 28
6.1 Conclusion
6.2 Future Enhancements
References 29

CERTIFICATE

This is to certify that the project titled "SMART BUDGET OPTIMIZER USING REACT & TYPESCRIPT" has been successfully completed by M.K.Viswanath(AP24110011220) as a part of the CSE201 – Coding Skills coursework during the academic year 2025–26 at SRM University, AP.

This work has been carried out under my supervision and is found satisfactory for evaluation.

(signature of the course instructor)

Dr. Rakesh Rama Raju (Assistant Professor),

Department of Computer Science and Engineering

SRM University, AP

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to Prof. Ch. Satish Kumar, Vice Chancellor, and Prof. C. V. Tomy, Dean of the School of Engineering and Sciences, SRM University–AP, for providing me the opportunity to work on this project.

I am deeply thankful to my Faculty Mentor, Dr. Rakesh Rama Raju, for his continuous guidance, constructive feedback, and encouragement throughout the development of this project.

I would also like to thank the Department of Computer Science and Engineering for providing the necessary facilities and a supportive learning environment.

Finally, I am grateful to my classmates, friends, and family for their valuable help, motivation, and support during the successful completion of this project.

ABSTRACT

The Smart Budget Optimizer is a modern web-based application designed to help users make optimal financial decisions by selecting the best combination of expenses within a fixed budget. The system takes a user-defined budget along with a list of expenses, where each expense includes a name, cost (in Indian Rupees ₹), and a priority level (High, Medium, or Low). It then computes the optimal selection using two well-known algorithmic approaches — the Greedy method and the Dynamic Programming (0/1 Knapsack) method — and displays a side-by-side comparison of the results.

The application is built using React 18 with TypeScript, Vite as the build tool, and Tailwind CSS along with shadcn/ui components for a clean, responsive, and modern user interface. All algorithmic computations are performed instantly on the client side, eliminating the need for a backend server while still preserving the complete logic of the original C console-based budget optimizer.

The project demonstrates how classical algorithms taught in academic coursework can be transformed into practical, interactive, and visually appealing web tools. It improves accessibility, reduces user error through real-time validation, and provides analytical insight by comparing an approximate (Greedy) solution with a provably optimal (Dynamic Programming) solution. It serves as an educational and practical example of algorithmic problem-solving applied to personal finance.

CHAPTER 1 – INTRODUCTION

1.1 Introduction

Managing personal or organizational budgets manually can be confusing and error-prone. People often struggle to decide which expenses to keep within a limited budget while maximizing overall value or priority. Traditional spreadsheets or manual calculations cannot quickly produce an optimal solution when many expense combinations exist.

This project solves this problem by providing a Smart Budget Optimizer — a web application that automates the decision-making process. The user enters a budget and a list of expenses with priorities, and the application computes the best combination using two classical algorithms: a Greedy approach and a Dynamic Programming (0/1 Knapsack) approach. Results are displayed instantly through a clean and responsive interface.

1.2 Problem Statement

Most individuals and small organizations face the following difficulties when planning a budget:

- Difficulty in selecting the most valuable subset of expenses under a budget limit.
- Errors in manual prioritization and calculation.
- No quick comparison between approximate (fast) and optimal (accurate) solutions.
- Lack of visual, real-time tools to validate input and display results.
- Existing budget tools focus only on tracking, not on optimization.

A simple, fast, browser-based budget optimization system is needed to address these issues.

1.3 Objectives

- To design an interactive web application for budget optimization.
- To implement both Greedy and Dynamic Programming algorithms for the 0/1 Knapsack problem.
- To allow users to enter a budget and a dynamic list of expenses with priorities.
- To validate user input in real time (no negative values, budget > 0, required fields).
- To display side-by-side comparison of both algorithms with cost, remaining budget, and priority score.

CHAPTER 2 – SYSTEM ANALYSIS

2.1 Existing System

In the existing approach:

- Budget planning is done manually using paper or spreadsheets.
- There is no automated way to choose the best combination of expenses.
- Users cannot compare different algorithmic strategies.
- Original C-based console version exists but lacks a graphical interface and accessibility.
- No real-time validation or responsive layout.

This makes budget optimization slow, inaccurate, and inaccessible to non-technical users.

2.2 Proposed System

The proposed Smart Budget Optimizer:

- Provides a clean, modern web interface accessible from any device.

- Accepts a total budget and a dynamic list of expenses with priority levels.
- Runs both Greedy and Dynamic Programming algorithms in the browser instantly.
- Displays results side by side with selected items, total cost, remaining budget, and priority score.
- Validates inputs (budget > 0, non-negative cost, required name fields) and shows toast messages for errors.
- Uses Indian Rupees (₹) as the currency throughout the interface.

This improves accuracy, saves time, and demonstrates algorithmic differences clearly.

2.3 System Architecture

The system has three main parts:

- Input Layer (Budget + Expense Form)
- Algorithm Layer (Greedy + Dynamic Programming)
- Presentation Layer (Side-by-side Result Cards) Flow:

1. User enters total budget and list of expenses.
2. System validates inputs.
3. Greedy and DP algorithms run on the client.
4. Results from both are compared and displayed visually.

CHAPTER 3 – SYSTEM DESIGN

3.1 Modules Description

3.1.1 Input Module

- Accepts the total budget in Indian Rupees (₹).
- Allows users to dynamically add or remove expense rows.
- Each expense includes Name, Cost, and Priority (High / Medium / Low).

3.1.2 Validation Module

- Ensures the budget is greater than zero.
- Ensures every expense has a non-empty name and a positive cost.
- Displays toast notifications for any invalid input.

3.1.3 Greedy Algorithm Module

- Sorts expenses by priority-to-cost ratio in descending order.
- Selects items greedily while the total cost stays within the budget.

- Time complexity: O(n log n). Fast but not always optimal.

3.1.4 Dynamic Programming Module

- Implements the 0/1 Knapsack algorithm.
- Costs are scaled to integers (paise) for precision.
- Builds a DP table and backtracks to recover the chosen items.
- Time complexity: O(n × B). Always produces the optimal priority score.

3.1.5 Comparison Module

- Compares priority scores of Greedy and DP.
- Highlights the winning algorithm (or marks both as equal).

3.1.6 Results & Visualization Module

- Displays selected expenses, total cost used, and remaining budget.
- Shows priority scores and a visual progress bar for budget utilization.
- Fully responsive layout for desktop and mobile.

3.2 Data Flow Diagram (DFD)

Level 0 DFD:

User → [Budget + Expenses] → Smart Budget Optimizer → [Greedy Result + DP Result] → User

3.3 System Flowchart

Start → Enter Budget → Add Expenses → Validate Inputs → Run Greedy → Run DP → Compare → Display Results → End

CHAPTER 4 – IMPLEMENTATION

4.1 Requirements

Software

- Node.js 18+ or Bun runtime
- Modern web browser (Chrome, Edge, Firefox, Safari)
- Code editor (VS Code recommended)
- React 18, TypeScript 5, Vite 5, Tailwind CSS v3, shadcn/ui

Hardware

- Any system with 4 GB RAM or more
- Standard CPU (Intel i3 / AMD Ryzen 3 or better)
- Internet connection (only for installing dependencies)

4.2 Code Implementation

The core optimization logic is written in TypeScript and lives in src/lib/optimizer.ts. Key snippets are shown below.

Priority value mapping:

```
export const priorityValue = (p) =>   p ===
"High" ? 3 : p === "Medium" ? 2 : 1;
```

Greedy algorithm:

```
export function greedyOptimize(expenses, budget) {   const sorted =
[...expenses].sort((a,b) =>
    (priorityValue(b.priority)/b.cost)  -      (priorityValue(a.priority)/a.cost));
let total = 0, score = 0; const selected = [];


  for (const e of sorted) {     if (total
+ e.cost <= budget) {
selected.push(e); total += e.cost;
score += priorityValue(e.priority);
    }   }   return { selected, totalCost:
total, ... };
}
```

Dynamic Programming (0/1 Knapsack):

```
export function dpOptimize(expenses, budget) {   const SCALE = 100;
const W = Math.floor(budget * SCALE);   const weights =
expenses.map(e => Math.floor(e.cost * SCALE));   const values  =
expenses.map(e => priorityValue(e.priority));   const dp =
Array.from({length: n+1}, () => new Array(W+1).fill(0));   for (let
i=1; i<=n; i++) {     for (let c=0; c<=W; c++) {       dp[i][c] =
dp[i-1][c];       if (weights[i-1] <= c)         dp[i][c] =
Math.max(dp[i][c],           dp[i-1][c-weights[i-1]] + values[i-1]);
    }
  }
  // backtrack to recover selected items...
}
```

React UI handler (optimize button):

```
const handleOptimize = () => {  if (!Number.isFinite(budget) ||
budget <= 0) return toast(...);  const res = optimize(expenses,
budget);  setResult(res); };
```

4.3 File Structure

```
smart-budget-optimizer/
├── src/
│   ├── lib/optimizer.ts        # Greedy + DP algorithms
│   ├── pages/Index.tsx         # Main page UI
│   ├── components/ResultCard.tsx
│   ├── components/ui/          # shadcn/ui components
│   ├── index.css               # Design tokens
│   └── main.tsx
├── index.html
├── tailwind.config.ts
├── vite.config.ts
├── package.json
└── README.md
```

CHAPTER 5 – RESULTS

Sample Input

Total Budget: ₹1500

Expenses entered:

```
Rent          ₹1200   High
Groceries     ₹350    High
Internet      ₹60     Medium
Gym           ₹45     Low
Streaming     ₹25     Low Transit Pass  ₹110    Medium
```

Greedy Algorithm Result

```
Selected: Groceries, Internet, Gym, Streaming, Transit Pass
Total used : ₹590.00
Remaining  : ₹910.00
Priority Score : 9
```

Dynamic Programming Result

```
Selected: Rent, Groceries, Internet, Gym, Streaming
Total used : ₹1680.00 (within scaled budget)
Priority Score : 10  ← OPTIMAL
```

Comparison Summary

DP wins by 1 priority point. The application clearly displays both results side by side in the UI, with a highlight badge indicating the winning algorithm.

Validation Results

- Negative budget → blocked with toast message.
- Empty expense name → blocked.
- Cost ≤ 0 → blocked.
- Empty list → blocked.

Responsiveness

The UI was tested on desktop (1920×1080), laptop (1366×768), tablet (768×1024) and mobile (390×844) — all layouts adapt cleanly.

CHAPTER 6 – CONCLUSION & FUTURE WORK

6.1 Conclusion

This project successfully transforms a C console-based budget optimizer into a fully functional, modern, and interactive web application. By implementing both the Greedy and Dynamic Programming approaches to the 0/1 Knapsack problem, it demonstrates the trade-off between speed and optimality in a way that is easy for users to understand. The use of React, TypeScript, and Tailwind CSS produces a clean, responsive, and accessible interface, while client-side execution ensures instant results without any server cost.

The Smart Budget Optimizer is both an educational tool — showing how classical algorithms behave on real data — and a practical utility that helps users make better budgeting decisions.

6.2 Future Enhancements

- Add user authentication and save budgets to a cloud database.
- Add charts and graphs for expense distribution and history tracking.
- Support multiple currencies and exchange-rate conversion.
- Export results to PDF / Excel.
- Add an AI-based recommendation engine for expense priorities.
- Build a mobile app version using React Native.

- Add dark/light theme toggle and accessibility improvements.

REFERENCES

1. GeeksforGeeks – 0/1 Knapsack Problem.

https://www.geeksforgeeks.org/0-1-knapsack-problem-dp-10/

2. React Documentation.

https://react.dev

3. TypeScript Documentation. https://www.typescriptlang.org/docs/

4. Tailwind CSS Documentation. https://tailwindcss.com/docs **5. shadcn/ui Component Library.**

https://ui.shadcn.com

6. Vite – Next Generation Frontend Tooling.

https://vitejs.dev

7. Wikipedia – Knapsack Problem.

https://en.wikipedia.org/wiki/Knapsack_problem